

Quinto projeto - Introdução à Física Computacional

Evandro Risso Venâncio¹

¹Instituto de Física de São Carlos - IFSC USP

risso.evandro@usp.br

Introdução

Este relatório apresenta o desenvolvimento e análise dos códigos implementados no quinto projeto da disciplina de Introdução à Física Computacional. O foco do projeto é o estudo das Leis de Kepler, com ênfase na simulação numérica de sistemas gravitacionais, como o movimento de planetas ao redor do Sol e o problema de três corpos.

Além disso, o projeto tem como objetivo aprender e implementar o método de Verlet, que é utilizado para simular órbitas.

Tarefa A

O primeiro exercício foi dividido em três partes. Na primeira, foi solicitado montar as expressões necessárias para implementar o método de Verlet e calcular as posições $x(t)$ e $y(t)$ de um planeta orbitando o Sol. Na segunda parte, foi determinado, por tentativa e erro, a velocidade inicial que cada planeta do sistema solar deveria ter para resultar em uma órbita circular. Por fim, na terceira parte, foram verificadas as Leis de Kepler para planetas com órbitas circulares.

O código abaixo implementa o método de Verlet para simular o movimento de um planeta ao redor do Sol (primeira parte):

```
1      program exal
2      implicit real*8 (a-h,o-z)
3
4      open(1, file="saida-al-#14668849.dat")
5
6      xt = 1.0
7      yt = 0.0
8
9      pi = acos(-1.0)
10
11     r = sqrt(xt**2 + yt**2)
12
13     vxt = 0.0
14     vyt = 2*pi*r
15
16     t = 0.0
17     delta_t = 1.0E-4
18     t_max = 1
19
20     x_old = xt - vxt*delta_t
21     y_old = yt - vyt*delta_t
22
23     do while (t .lt. t_max)
24
25         r = sqrt(xt**2 + yt**2)
```

```

26
27     ax = -(4*pi**2*xt)/r**3
28     ay = -(4*pi**2*yt)/r**3
29
30     xt2 = 2.0*xt - x_old + (ax*delta_t**2)
31     yt2 = 2.0*yt - y_old + (ay*delta_t**2)
32
33     x_old = xt
34     y_old = yt
35     yt = yt2
36     xt = xt2
37
38     t = t + delta_t
39
40     write(1, *) t, xt, yt, r
41
42 end do
43
44 close(1)
45
46 end program exal

```

Neste código, a integração das equações de movimento é realizada utilizando o método de Verlet. As posições iniciais do planeta ($x = 1.0$, $y = 0.0$) e sua velocidade inicial ($v_x = 0.0$, $v_y = 2\pi r$) são configuradas para uma órbita circular. A constante gravitacional $GMS = 4\pi^2$ foi usada nas unidades astronômicas. O passo de tempo ($\Delta t = 10^{-4}$) garante precisão na integração.

O loop principal simula a órbita do planeta Terra, atualizando as posições em pequenos passos de tempo:

- Calcula a distância atual ($r = \sqrt{x^2 + y^2}$).
- Determina as acelerações a_x e a_y de acordo com a gravidade ($a_x = -\frac{4\pi^2 x}{r^3}$, $a_y = -\frac{4\pi^2 y}{r^3}$).
- Atualiza as novas posições x e y utilizando o método de Verlet:

$$x_{t+1} = 2x_t - x_{t-1} + a_x(\Delta t)^2$$

$$y_{t+1} = 2y_t - y_{t-1} + a_y(\Delta t)^2$$

- Ajusta os valores antigos (x_{old} e y_{old}) para os próximos passos.

O gráfico da simulação ficou como:

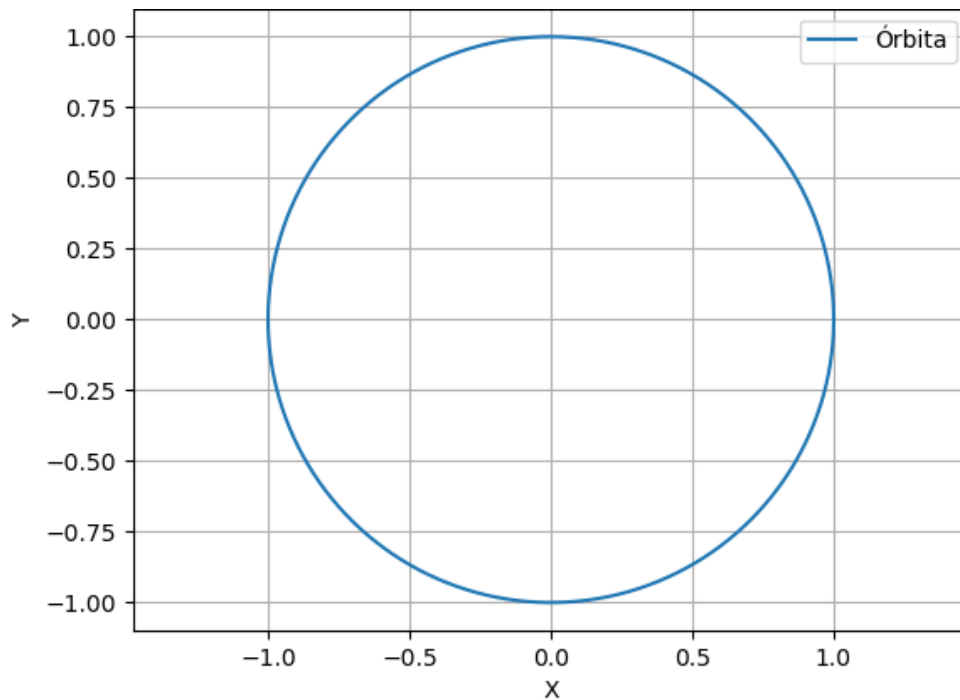


Figura 1. Órbita circular da Terra

Na segunda parte do exercício, foi solicitado determinar, por tentativa e erro, a velocidade inicial que cada planeta do sistema solar deveria ter para gerar uma órbita circular, e calcular a razão T^2/R^3 para verificar a Terceira Lei de Kepler. Para isso, foi implementado um código que simula o movimento dos planetas ao redor do Sol, considerando suas distâncias médias (em unidades astronômicas) e velocidades ajustadas para trajetórias circulares.

O código desenvolvido está apresentado abaixo:

```

1  program exa2
2      implicit real*8 (a-h,o-z)
3
4      dimension raio_ua(9), vx(9), vy(9), ax(9), ay(9)
5      dimension x(9), y(9), x_ant(9), y_ant(9)
6      character*8 planetas(9)
7
8      open(1, file='saida-a2-#14668849.dat')
9      open(2, file='saida-a2-#14668849-2.dat')
10
11      pi = acos(-1.0)
12
13      raio_ua = (/0.39, 0.72, 1.0, 1.52, 5.2, 9.58,
14 & 19.22, 30.05, 39.48/)
15      planetas = (/ "Mercurio", "Venus ", "Terra ", "Marte ",
16 & "Jupiter ", "Saturno ", "Urano ", "Netuno ", "Plutao " /)
17
18      write(2, *) "Planeta", "Raio (UA)", "Periodo (anos)", "T^2/R^3"
19
20      do i = 1, 9
21          x(i) = raio_ua(i)
22          y(i) = 0.0

```

```

23
24     vx(i) = 0.0
25     vy(i) = (2.0*pi)/sqrt(raio_ua(i))
26
27     r = sqrt(x(i)**2 + y(i)**2)
28     ax(i) = -((4.0*(pi**2)*x(i))/r**3)
29     ay(i) = -((4.0*(pi**2)*y(i))/r**3)
30
31     t = 0.0
32     delta_t = 1.0E-4
33     t_max = raio_ua(i)**1.5
34
35     x_ant(i) = x(i) - (vx(i)*delta_t) + (0.5*ax(i)*delta_t**2)
36     y_ant(i) = y(i) - (vy(i)*delta_t) + (0.5*ay(i)*delta_t**2)
37
38     do while (t .lt. t_max)
39
40         r = sqrt(x(i)**2 + y(i)**2)
41
42         ax(i) = -((4.0*(pi**2)*x(i))/r**3)
43         ay(i) = -((4.0*(pi**2)*y(i))/r**3)
44
45         x_new = 2.0*x(i) - x_ant(i) + (ax(i) * delta_t**2)
46         y_new = 2.0*y(i) - y_ant(i) + (ay(i) * delta_t**2)
47
48         x_ant(i) = x(i)
49         y_ant(i) = y(i)
50
51         x(i) = x_new
52         y(i) = y_new
53
54         r = sqrt(x(i)**2 + y(i)**2)
55
56         !write(1, *) t, x(i), y(i), r
57
58         t = t + delta_t
59
60     end do
61
62     raio = (t_max**2)/(raio_ua(i)**3)
63     write(2, *) planetas(i), raio_ua(i), t_max, raio
64
65 end do
66
67 close(1); close(2)
68 end program exa2

```

- A posição inicial (x) de cada planeta é definida como o raio médio de sua órbita ($raio_ua$), enquanto $y = 0$.
- A velocidade inicial v_y é calculada para garantir uma órbita circular, utilizando a relação $(2\pi)/\sqrt{raio_ua}$.
- As acelerações iniciais são calculadas com base na força gravitacional.
- As posições anteriores (x_{ant} e y_{ant}) são estimadas com o método de Euler modificado.

- Durante o loop principal, as novas posições são calculadas usando o método de Verlet, enquanto as forças e acelerações são atualizadas a cada passo.
- Após o término da simulação para cada planeta, o período $T = t_{\max}$ e a razão T^2/R^3 são calculados. Os resultados são armazenados em um arquivo.

O código gera dois arquivos de saída: um com as posições calculadas ao longo do tempo (`saida-a2-14668849.dat`) e outro com os valores de T^2/R^3 para cada planeta (`saida-a2-14668849-2.dat`).

Para exemplificar, abaixo estão as órbitas de Marte e Saturno, mostrando que elas ficaram circulares:

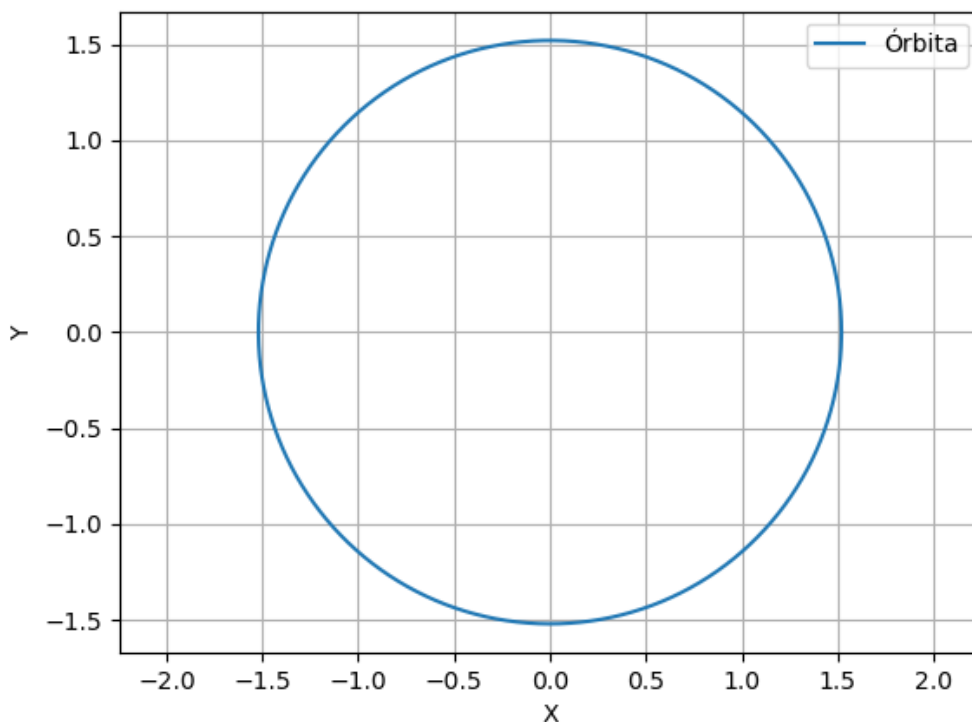


Figura 2. Órbita circular de Marte

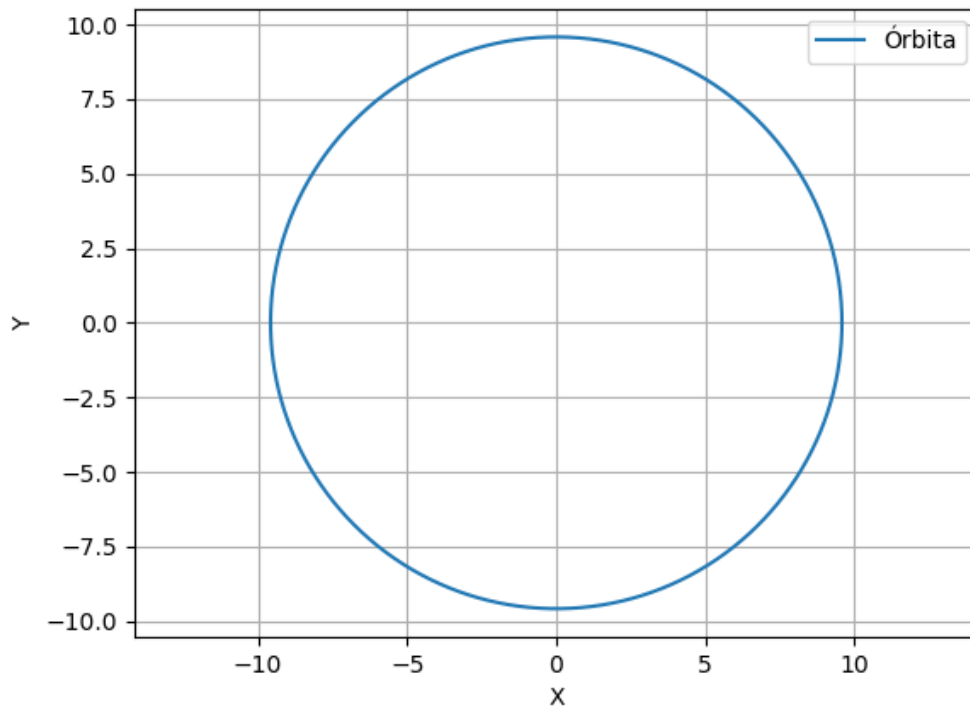


Figura 3. Órbita circular de Saturno

A tabela criada para as razões de T^2/R^3 foi a seguinte:

Tabela 1. Relação entre T^2/R^3 para os planetas do Sistema Solar

Planeta	Raio (UA)	Período (anos)	T^2/R^3
Mercúrio	0.38999998569488525	0.24355490853722572	1.0000000000000002
Vênus	0.72000002861022949	0.61094029536005456	1.0000000000000002
Terra	1.0000000000000000	1.0000000000000000	1.0000000000000000
Marte	1.5199999809265137	1.8739818216295090	0.9999999999999999
Júpiter	5.1999998092651367	11.857823768617429	1.0000000000000000
Saturno	9.5799999237060547	29.651608573466117	1.0000000000000002
Urano	19.219999313354492	84.261667957869022	0.9999999999999999
Netuno	30.049999237060547	164.72772401205376	0.9999999999999999
Plutão	39.479999542236328	248.06512300501248	1.0000000000000000

A razão T^2/R^3 foi aproximadamente constante igual a 1 para todos os planetas, conforme a Terceira Lei de Kepler. Esta constância valida o modelo gravitacional utilizado, além de demonstrar a precisão do método de Verlet aplicado à simulação de sistemas orbitais.

Por fim na terceira parte do exercício, foi solicitado verificar as órbitas não circulares dos planetas do Sistema Solar, considerando suas excentricidades. O objetivo principal foi simular as trajetórias elípticas, determinando os semi-eixos maiores e validando a Terceira Lei de Kepler para essas órbitas. O código desenvolvido está descrito abaixo:

```

1  program exa3
2      implicit real*8 (a-h, o-z)
3
4      dimension raio_ua(9), vx(9), vy(9), ax(9), ay(9)
5      dimension x(9), y(9), x_ant(9), y_ant(9)
6      dimension semi_eixo(9), excentricidade(9)
7      character*8 planetas(9)
8
9      open(1, file='saida-a3-#14668849.dat')
10     open(2, file='saida-a3-#14668849-2.dat')
11
12     pi = acos(-1.0)
13
14     raio_ua = (/0.39, 0.72, 1.0, 1.52, 5.2, 9.58,
15 & 19.22, 30.05, 39.48/)
16     excentricidade = (/0.206, 0.007, 0.017, 0.093,
17 & 0.048, 0.056, 0.046, 0.010, 0.248/)
18
19     planetas = (/ "Mercurio", "Venus  ", "Terra  ", "Marte  ",
20 & "Jupiter ", "Saturno ", "Urano  ", "Netuno ", "Plutao " /)
21
22     write(2, *) "Planeta", "Raio (UA)", "Periodo (anos)", "T^2/R^3"
23
24     do i = 1, 9
25         semi_eixo(i) = raio_ua(i)/(1.0 - excentricidade(i))
26     end do
27
28     do i = 1, 9
29         x(i) = raio_ua(i)
30         y(i) = 0.0
31
32         vx(i) = 0.0
33         vy(i) = sqrt((4.0*pi**2*(2.0/raio_ua(i) - 1.0/semi_eixo(i))))
34
35         r = sqrt(x(i)**2 + y(i)**2)
36         ax(i) = -((4.0*(pi**2)*x(i))/r**3)
37         ay(i) = -((4.0*(pi**2)*y(i))/r**3)
38
39         t = 0.0
40         delta_t = 1.0E-3
41         t_max = semi_eixo(i)**1.5
42
43         x_ant(i) = x(i) - (vx(i)*delta_t) + (0.5*ax(i)*delta_t**2)
44         y_ant(i) = y(i) - (vy(i)*delta_t) + (0.5*ay(i)*delta_t**2)
45
46         do while (t .lt. t_max)
47
48             r = sqrt(x(i)**2 + y(i)**2)
49
50             ax(i) = -((4.0*(pi**2)*x(i))/r**3)
51             ay(i) = -((4.0*(pi**2)*y(i))/r**3)
52
53             x_new = 2.0*x(i) - x_ant(i) + (ax(i) * delta_t**2)
54             y_new = 2.0*y(i) - y_ant(i) + (ay(i) * delta_t**2)
55
56             x_ant(i) = x(i)

```

```

57         y_ant(i) = y(i)
58
59         x(i) = x_new
60         y(i) = y_new
61
62         r = sqrt(x(i)**2 + y(i)**2)
63
64         !write(1, *) t, x(i), y(i), r
65
66         t = t + delta_t
67
68     end do
69
70     raio = (t_max**2) / (raio_ua(i)**3)
71     write(2, *) planetas(i), raio_ua(i), t_max, raio
72
73 end do
74
75 close(1)
76
77 end program exa3

```

- O semi-eixo maior de cada órbita foi calculado a partir do raio médio da órbita (*raio_ua*) e da excentricidade (*e*), utilizando a relação:

$$a = \frac{r}{1 - e}.$$

- A velocidade inicial foi ajustada considerando a relação energética para órbitas elípticas:

$$v_y = \sqrt{4\pi^2 \left(\frac{2}{r} - \frac{1}{a} \right)}.$$

- O método de Verlet foi utilizado novamente para integrar as equações de movimento ao longo do tempo, considerando as forças gravitacionais centradas no Sol. O passo de tempo ficou como $\Delta t = 10^{-3}$.
- Após a simulação, o período orbital (T) foi determinado pela relação:

$$T = a^{3/2},$$

e a razão T^2/R^3 foi calculada para verificar a consistência com a Terceira Lei de Kepler.

Os gráficos de Mercúrio e Júpiter estão representados abaixo para exemplificar as órbitas elípticas fechadas.

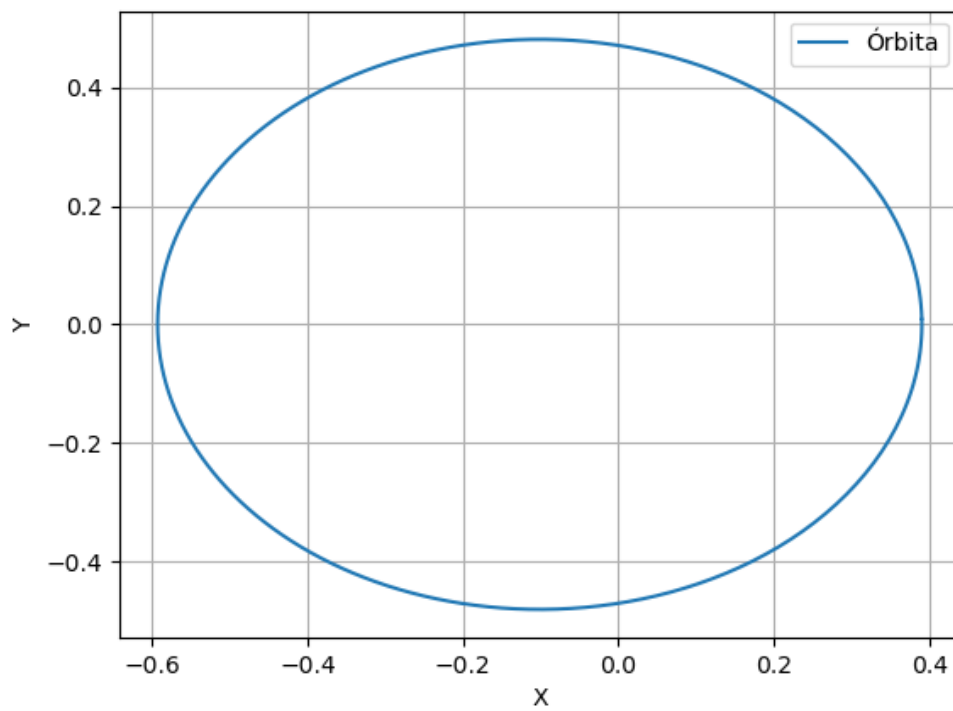


Figura 4. Órbita elíptica - Mercúrio

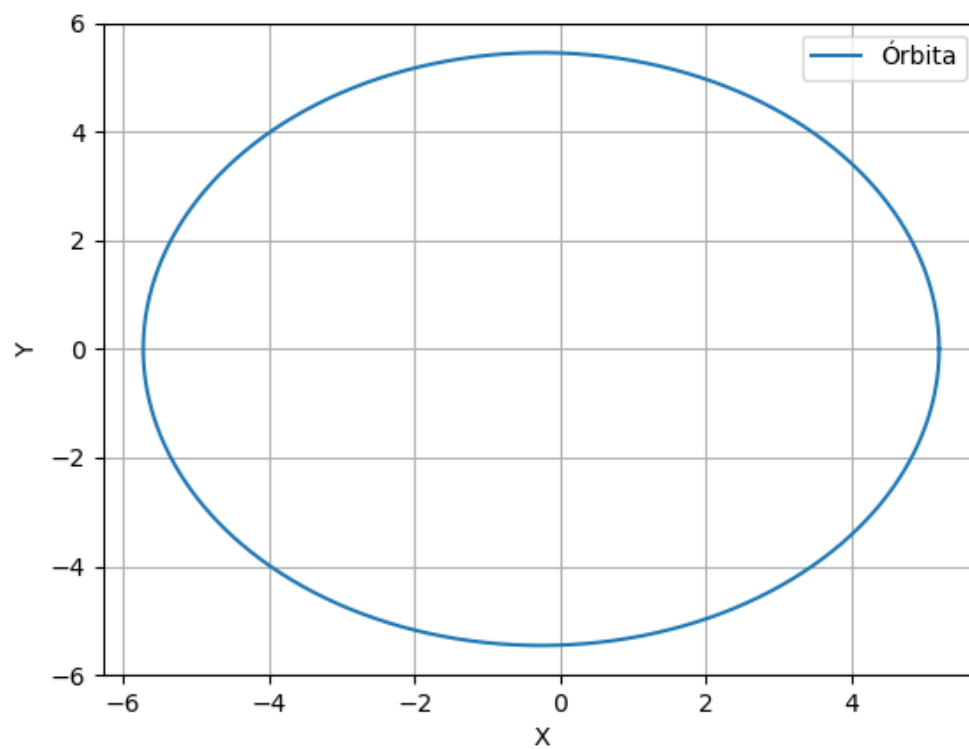


Figura 5. Órbita elíptica - Júpiter

Para verificar a relação da razão T^2/R^3 a tabela abaixo foi criada.

Tabela 2. Relação entre T^2/R^3 para órbitas elípticas dos planetas do Sistema Solar

Planeta	Raio (UA)	Período (anos)	T^2/R^3
Mercúrio	0.38999998569488525	0.34424380480156835	1.9977378274134994
Vênus	0.72000002861022949	0.61741176083650362	1.0212974670379436
Terra	1.00000000000000000	1.0260528330896603	1.0527844162913185
Marte	1.5199999809265137	2.1694739751730636	1.3402263182195642
Júpiter	5.1999998092651367	12.765845066012188	1.1590152635852076
Saturno	9.5799999237060547	32.328850892360457	1.1887321797317127
Urano	19.219999313354492	90.428950332074137	1.1517411293118849
Netuno	30.049999237060547	167.22989070467551	1.0306101514303059
Plutão	39.479999542236328	380.39862989609122	2.3515080574863569

As órbitas simuladas demonstraram o comportamento esperado para trajetórias elípticas, considerando as excentricidades fornecidas. No entanto, os valores de T^2/R^3 apresentaram desvios significativos da constância prevista pela Terceira Lei de Kepler, especialmente para Mercúrio (1.9977) e Plutão (2.3515).

Esses desvios podem ser atribuídos a aproximações no modelo numérico ou a limitações no passo de tempo Δt . Ainda assim, para planetas com excentricidades menores, como Vênus (1.0213) e Netuno (1.0306), os valores de T^2/R^3 estão mais próximos da unidade, indicando maior consistência.

Tarefa B

Neste exercício, o objetivo foi explorar o problema dos três corpos, analisando as interações gravitacionais entre a Terra, o Sol e Júpiter. A tarefa foi dividida em três partes. Na primeira, utilizou-se o método de Verlet para simular o movimento desses corpos, demonstrando que, diferentemente do problema de dois corpos, a órbita terrestre não é mais periódica devido à influência gravitacional de Júpiter.

Na segunda parte, investigou-se como o aumento da massa de Júpiter, multiplicando-a por 10 e 1000 vezes, altera as dinâmicas orbitais do sistema. Por fim, na terceira parte, foi analisado as órbitas de três asteroides sob a influência gravitacional combinada de Júpiter e do Sol, avaliando os efeitos dessas forças no comportamento orbital.

Na primeira parte, o objetivo foi simular o problema dos três corpos considerando a interação gravitacional entre o Sol, a Terra e Júpiter. Para isso, foi utilizado o método de Verlet, que integra as equações de movimento para calcular as trajetórias dos corpos ao longo do tempo.

O código implementado segue a lógica descrita abaixo:

```
1  program exb1
2      implicit real*8 (a-h,o-z)
3
4      open(1, file="terra.dat")
5      open(2, file="jupiter.dat")
6
7      pi = acos(-1.0)
```

```

8      am_s = 1.989E30
9      am_t = 5.972E24
10     am_j = 1.898E27
11
12     ! Posicao Inicial - SOL
13     xs = 0.0
14     ys = 0.0
15
16     ! Posicao Inicial e velocidades iniciais - TERRA
17     xt = 1.0
18     yt = 0.0
19     vxt = 0.0
20     vyt = (2.0*pi)/sqrt(xt)
21
22     ! Posicao Inicial e velocidades iniciais - JUPITER
23     xj = 5.2
24     yj = 0.0
25     vxj = 0.0
26     vyj = (2.0*pi)/sqrt(xj)
27
28     t = 0.0
29     delta_t = 1.0E-4
30     t_max = xj**1.5
31
32     ! Posi o em rela o aos planetas
33     rt = sqrt((xt-xs)**2 + (yt-xs)**2)
34     rj = sqrt((xj-xs)**2 + (yj-xs)**2)
35     rtj = sqrt((xt-xj)**2 + (yt-yj)**2)
36
37     ! Acelera o da Terra em rela o ao Sol e a Jupiter - X
38     axt = -(4.0*(pi**2)*xt)/rt**3
39     & - (4.0*(pi**2)*(am_j/am_s)*(xt-xj))/rtj**3
40
41     ! Acelera o da Terra em rela o ao Sol e a Jupiter - Y
42     ayt = -(4.0*(pi**2)*yt)/rt**3
43     & - (4.0*(pi**2)*(am_j/am_s)*(yt-yj))/rtj**3
44
45     ! Posi o anterior Terra
46     x_ant_t = xt - (vxt*delta_t) + (0.5*axt*delta_t**2)
47     y_ant_t = yt - (vyt*delta_t) + (0.5*ayt*delta_t**2)
48
49     ! Acelera o de Jupiter em rela o ao Sol e a Terra - X
50     axj = -(4.0*(pi**2)*xj)/rj**3
51     & - (4.0*(pi**2)*(am_t/am_s)*(xj-xt))/rtj**3
52
53     ! Acelera o de Jupiter em rela o ao Sol e a Terra - Y
54     ayj = -(4.0*(pi**2)*yj)/rj**3
55     & - (4.0*(pi**2)*(am_t/am_s)*(yj-yt))/rtj**3
56
57     ! Posi o anterior Jupiter
58     x_ant_j = xj - (vxj*delta_t) + (0.5*axj*delta_t**2)
59     y_ant_j = yj - (vyj*delta_t) + (0.5*ayj*delta_t**2)
60
61     do while (t .le. t_max)
62
63         rt = sqrt((xt-xs)**2 + (yt-xs)**2)

```

```

64      rj = sqrt((xj-xs)**2 + (yj-xs)**2)
65      rtj = sqrt((xt-xj)**2 + (yt-yj)**2)
66
67      axt = -(4.0*(pi**2)*xt)/rt**3
68      &    - (4.0*(pi**2)*(am_j/am_s)*(xt-xj))/rtj**3
69
70      ayt = -(4.0*(pi**2)*yt)/rt**3
71      &    - (4.0*(pi**2)*(am_j/am_s)*(yt-yj))/rtj**3
72
73      axj = -(4.0*(pi**2)*xj)/rj**3
74      &    - (4.0*(pi**2)*(am_t/am_s)*(xj-xt))/rtj**3
75
76      ayj = -(4.0*(pi**2)*yj)/rj**3
77      &    - (4.0*(pi**2)*(am_t/am_s)*(yj-yt))/rtj**3
78
79      x_new_t = 2.0*xt - x_ant_t + (axt * delta_t**2)
80      y_new_t = 2.0*yt - y_ant_t + (ayt * delta_t**2)
81
82      x_new_j = 2.0*xj - x_ant_j + (axj * delta_t**2)
83      y_new_j = 2.0*yj - y_ant_j + (ayj * delta_t**2)
84
85      x_ant_t = xt
86      y_ant_t = yt
87      y_ant_j = yj
88      x_ant_j = xj
89
90      xt = x_new_t
91      yt = y_new_t
92      xj = x_new_j
93      yj = y_new_j
94
95      write(1, *) xt, yt
96      write(2, *) xj, yj
97
98      t = t + delta_t
99  end do
100
101  close(1); close(2)
102
103  end program exb1

```

- As posições iniciais do Sol foram fixadas na origem ($x_s = 0, y_s = 0$).
- A Terra ($x_t = 1$) e Júpiter ($x_j = 5.2$) foram posicionados de acordo com suas distâncias médias do Sol, com velocidades ajustadas para órbitas quase circulares.
- A aceleração de cada corpo é determinada pela soma vetorial das forças gravitacionais. Por exemplo, a aceleração da Terra é dada por:

$$a_x^T = -\frac{4\pi^2 x_T}{r_T^3} - \frac{4\pi^2 M_J / M_S (x_T - x_J)}{r_{TJ}^3}.$$

- Fórmulas análogas são usadas para o eixo y da Terra e para os componentes de Júpiter.
- As posições de cada corpo são atualizadas iterativamente utilizando o método de Verlet:

$$x_{\text{novo}} = 2x - x_{\text{anterior}} + a_x \Delta t^2.$$

O gráfico resultante foi:

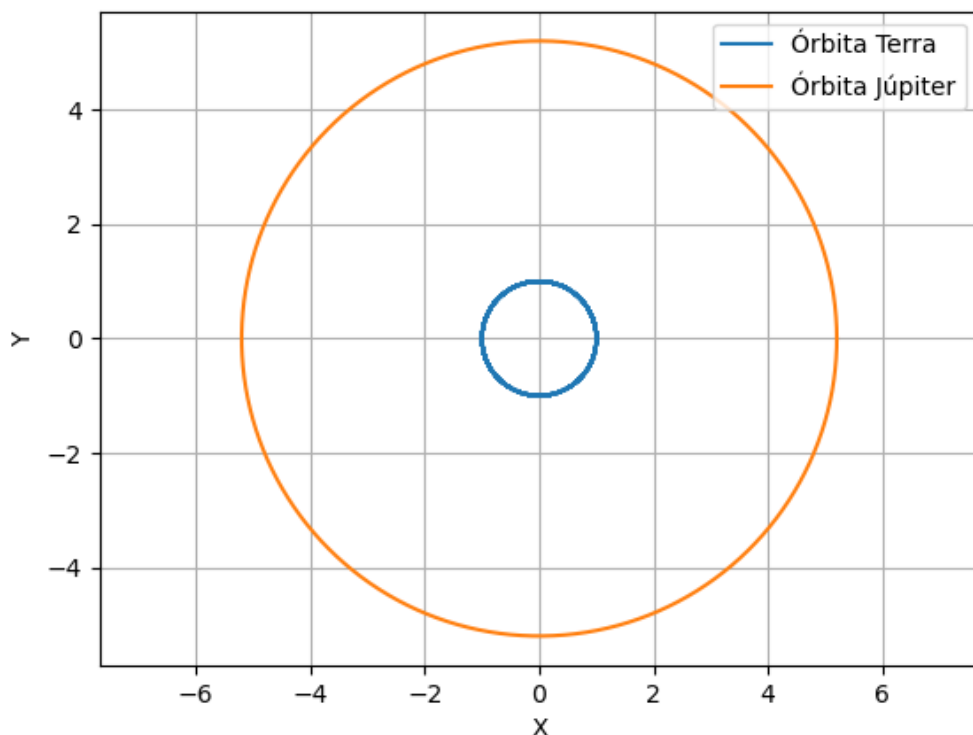


Figura 6. Órbitas - Sol, Júpiter e Terra

Com o método implementado, espera-se observar que a órbita da Terra não é mais periódica devido à influência gravitacional de Júpiter. As perturbações causadas por Júpiter deverão alterar ligeiramente a trajetória terrestre, diferentemente do comportamento observado no problema de dois corpos.

Como a influência de Júpiter é muito pequena em relação a influência do Sol, não é possível verificar claramente no gráfico as perturbações que Júpiter causa na Terra.

Na segunda parte apenas foi modificada a massa de Júpiter, a primeira imagem representa Júpiter com 10x de massa e na segunda 1000x.

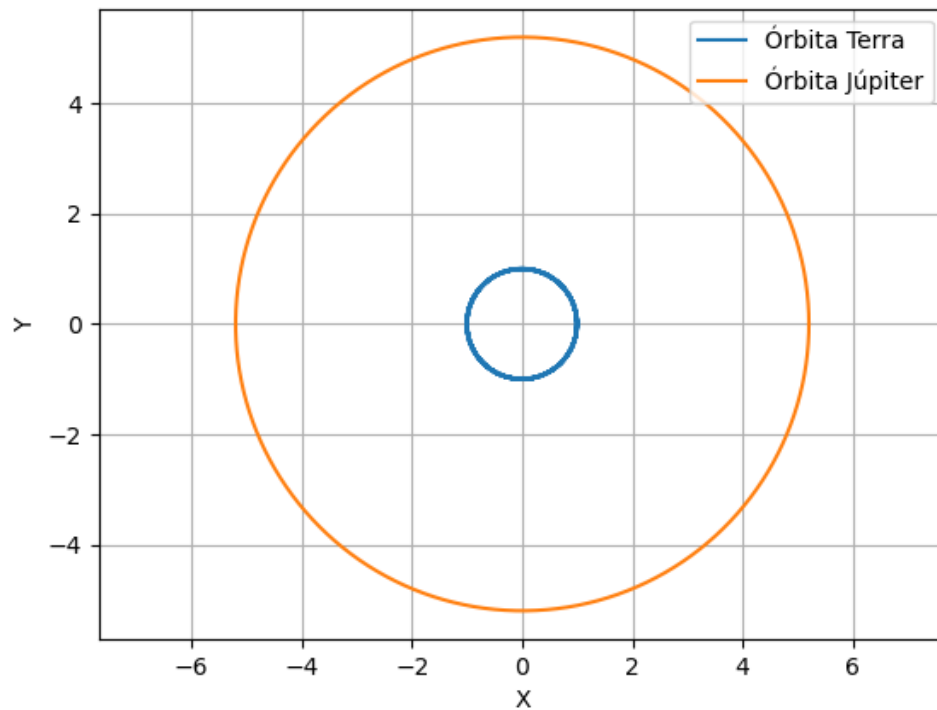


Figura 7. Órbita - Terra, Sol, Júpiter (10x massa)

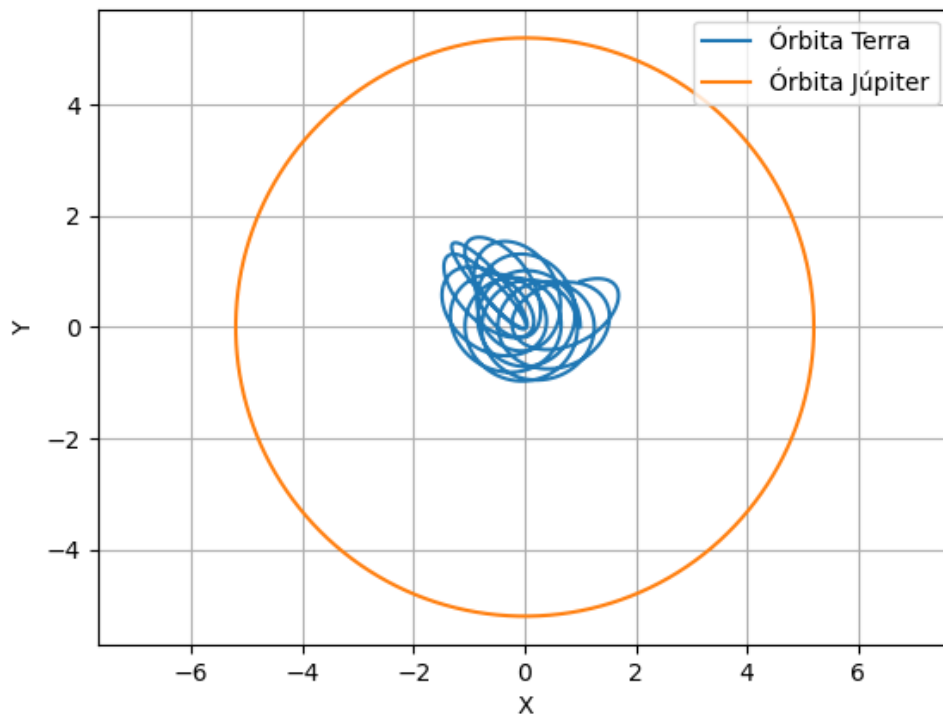


Figura 8. Órbita - Terra, Sol, Júpiter (1000x massa)

Percebe-se que quando a massa de Júpiter aumenta 10x, ainda assim tem pouca influência na Terra. Porém quando está 1000x maior, a força gravitacional dele será maior que o do Sol, afetando a órbita da Terra.

Por fim, na última tarefa, teve como objetivo verificar a órbita de 3 asteroides com influências de Júpiter e do Sol. Os dados de cada asteroide está abaixo na tabela:

Tabela 3. Dados iniciais dos asteroides simulados

Objeto	Raio (UA)	Velocidade (UA/ano)
Asteroide I	3.000	3.628
Asteroide II	3.276	3.471
Asteroide III	3.700	3.267

Para realizar as órbitas, foram utilizadas as posições iniciais e velocidades dos asteroides, juntamente com o método de Verlet para integrar as equações de movimento. O código foi implementado para calcular as trajetórias dos asteroides e de Júpiter ao longo do tempo.

```
1  program exb3
2      implicit real*8 (a-h, o-z)
3
4      dimension aster_raio(3), aster_vel(3), r_ast_s(3), r_ast_j(3)
5      dimension x(3), y(3), ax(3), ay(3), x_ant(3), y_ant(3)
6
7      open(1, file="asteroides1.dat")
8      open(2, file="asteroides2.dat")
9      open(3, file="asteroides3.dat")
10
11      am_j = 1.898E27
12      am_s = 1.989E30
13      pi = acos(-1.0)
14
15      xs = 0.0
16      ys = 0.0
17
18      xj = 5.2
19      yj = 0.0
20      vxj = 0.0
21      vyj = 2.76
22
23      aster_raio = (/ 3.0, 3.28, 3.7 /)
24      aster_vel = (/ 3.63, 3.47, 3.27 /)
25
26      xj_old = xj - vxj * delta_t
27      yj_old = yj - vyj * delta_t
28
29      t = 0.0
30      delta_t = 1.0E-5
31      t_max = 500.0
32
33      do i = 1, 3
34          x(i) = aster_raio(i)
35          y(i) = 0.0
```

```

36
37     r_ast_s(i) = sqrt((x(i))**2 + (y(i))**2)
38     r_ast_j(i) = sqrt((x(i)-xj)**2 + (y(i)- yj)**2)
39
40     ax(i) = -(4.0*(pi**2)*(x(i)))/r_ast_s(i)**3
41 &     - (4.0*(pi**2)*(am_j/am_s)*(x(i)-xj))/r_ast_j(i)**3
42
43     ay(i) = -(4.0*(pi**2)*(y(i)))/r_ast_s(i)**3
44 &     - (4.0*(pi**2)*(am_j/am_s)*(y(i)-yj))/r_ast_j(i)**3
45
46     x_ant(i) = x(i)+(0.5*ax(i)*delta_t**2)
47     y_ant(i) = y(i)-(aster_vel(i)*delta_t)+(0.5*ay(i)*delta_t**2)
48 end do
49
50 do while (t .le. t_max)
51
52     r = sqrt(xj**2 + yj**2)
53     ajx = -((4.0*(pi**2)*xj)/r**3)
54     ajy = -((4.0*(pi**2)*yj)/r**3)
55
56
57     xj_new = 2.0*xj - xj_old + (ajx * delta_t**2)
58     yj_new = 2.0*yj - yj_old + (ajy * delta_t**2)
59
60     xj_old = xj
61     yj_old = yj
62     xj = xj_new
63     yj = yj_new
64
65     do i = 1, 3
66         r_ast_s(i) = sqrt((x(i)-xs)**2 + (y(i)-ys)**2)
67         r_ast_j(i) = sqrt((x(i)-xj)**2 + (y(i)-yj)**2)
68
69         ax(i) = -(4.0*(pi**2)*(x(i)-xs))/r_ast_s(i)**3
70 &         - (4.0*(pi**2)*(am_j/am_s)*(x(i)-xj))/r_ast_j(i)**3
71
72         ay(i) = -(4.0*(pi**2)*(y(i)-ys))/r_ast_s(i)**3
73 &         - (4.0*(pi**2)*(am_j/am_s)*(y(i)-yj))/r_ast_j(i)**3
74
75         x_new = 2.0*x(i) - x_ant(i) + (ax(i)*delta_t**2)
76         y_new = 2.0*y(i) - y_ant(i) + (ay(i)*delta_t**2)
77
78         x_ant(i) = x(i)
79         y_ant(i) = y(i)
80         x(i) = x_new
81         y(i) = y_new
82
83         if (i == 1) then
84             write(1, *) x(i), y(i)
85         else if (i == 2) then
86             write(2, *) x(i), y(i)
87         else
88             write(3, *) x(i), y(i)
89         end if
90
91     end do

```

```

92     t = t + delta_t
93     end do
94
95     close(1); close(2); close(3)
96 end program exb3

```

- As posições iniciais de Júpiter ($x_j = 5.2$) e dos asteroides ($x_1 = 3.0, x_2 = 3.28, x_3 = 3.7$) foram definidas com base nos dados fornecidos.
- As velocidades iniciais dos asteroides foram ajustadas para refletir suas condições orbitais ($v_1 = 3.63, v_2 = 3.47, v_3 = 3.27$).
- A aceleração de cada asteroide foi calculada considerando a influência gravitacional do Sol e de Júpiter:

$$a_x = -\frac{4\pi^2 x}{r_S^3} - \frac{4\pi^2 M_J / M_S (x - x_J)}{r_J^3}.$$

- As posições dos asteroides e de Júpiter foram atualizadas iterativamente utilizando o método de Verlet:

$$x_{\text{novo}} = 2x - x_{\text{anterior}} + a_x \Delta t^2.$$

O gráfico das órbitas dos asteroides estão mostradas abaixo:

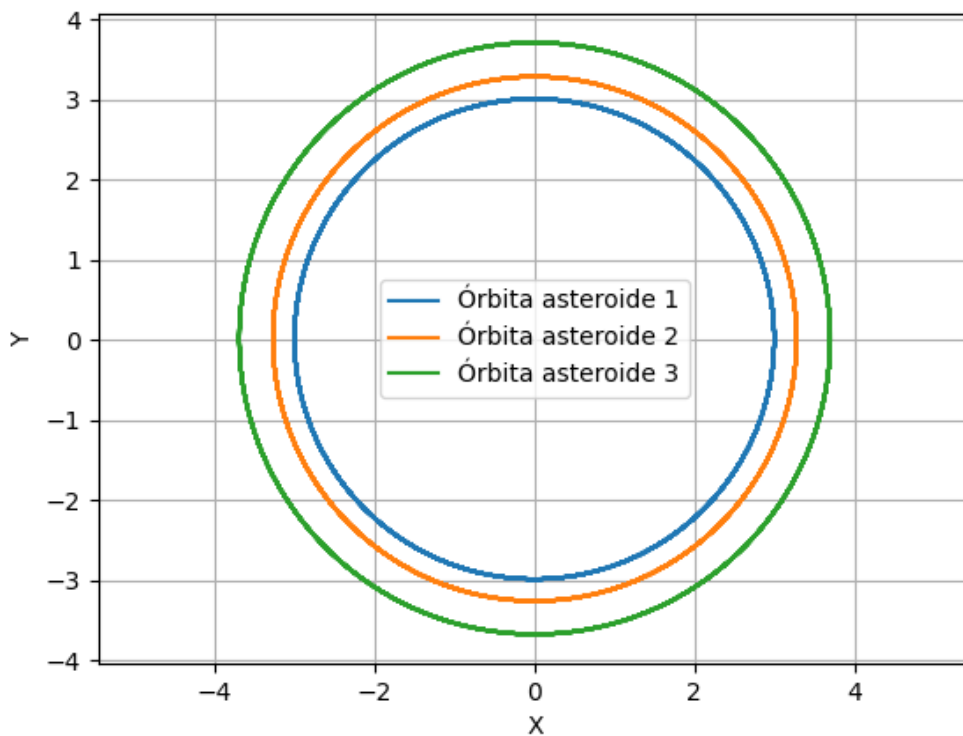


Figura 9. Órbita dos asteroides

Os asteroides próximos à órbita de Júpiter podem apresentar perturbações significativas em suas trajetórias, especialmente devido à ressonância gravitacional. Este comportamento é consistente com as lacunas de Kirkwood observadas no cinturão de asteroides, onde regiões específicas têm menos objetos devido às influências gravitacionais

de Júpiter.

Acredito que por causa do pequeno tempo de simulação, não foi possível observar corretamente essas perturbações.